ESP32-SecureMQTT

Mise en place d'un accès distant sécurisé via MQTTs/TLS

Auteur : Nissa Karadag

Décembre 2025

TP IoT Sécurisé — BTS / Licence Pro / BUT Réseaux

1. Objectifs et contexte

L'objectif de ce TP est de mettre en place une infrastructure permettant d'accéder à distance à un microcontrôleur ESP32 de manière sécurisée, sans l'exposer directement sur Internet.

Problématique

L'ESP32 est un microcontrôleur aux ressources limitées (CPU, RAM) qui n'est pas conçu pour être un point d'entrée réseau. Les contraintes NAT et Wi-Fi instable rendent l'accès direct difficile et risqué.

Stack technique

Composant	Technologie
Microcontrôleur	ESP32 (MicroPython v1.26)
Serveur broker	VM Ubuntu 22.04 (VirtualBox, mode bridge)
Protocole	MQTT over TLS (MQTTs, port 8883)
Broker MQTT	Mosquitto 2.0.18
Chiffrement	TLS via certificats OpenSSL auto-signés
Authentification	Username/password + ACL
Pare-feu	UFW
Administration	SSH (port 22)
IDE ESP32	Thonny (MicroPython)

2. Mise en place de la VM Ubuntu

2.1 Configuration réseau

La machine virtuelle est configurée en mode bridge dans VirtualBox. Elle obtient une adresse IP directement depuis le réseau local (192.168.1.183), ce qui la rend accessible comme une machine physique sans redirection de ports.

Test de connectivité réseau : ping vers google.com et 8.8.8.8 (0% de perte de paquets) :

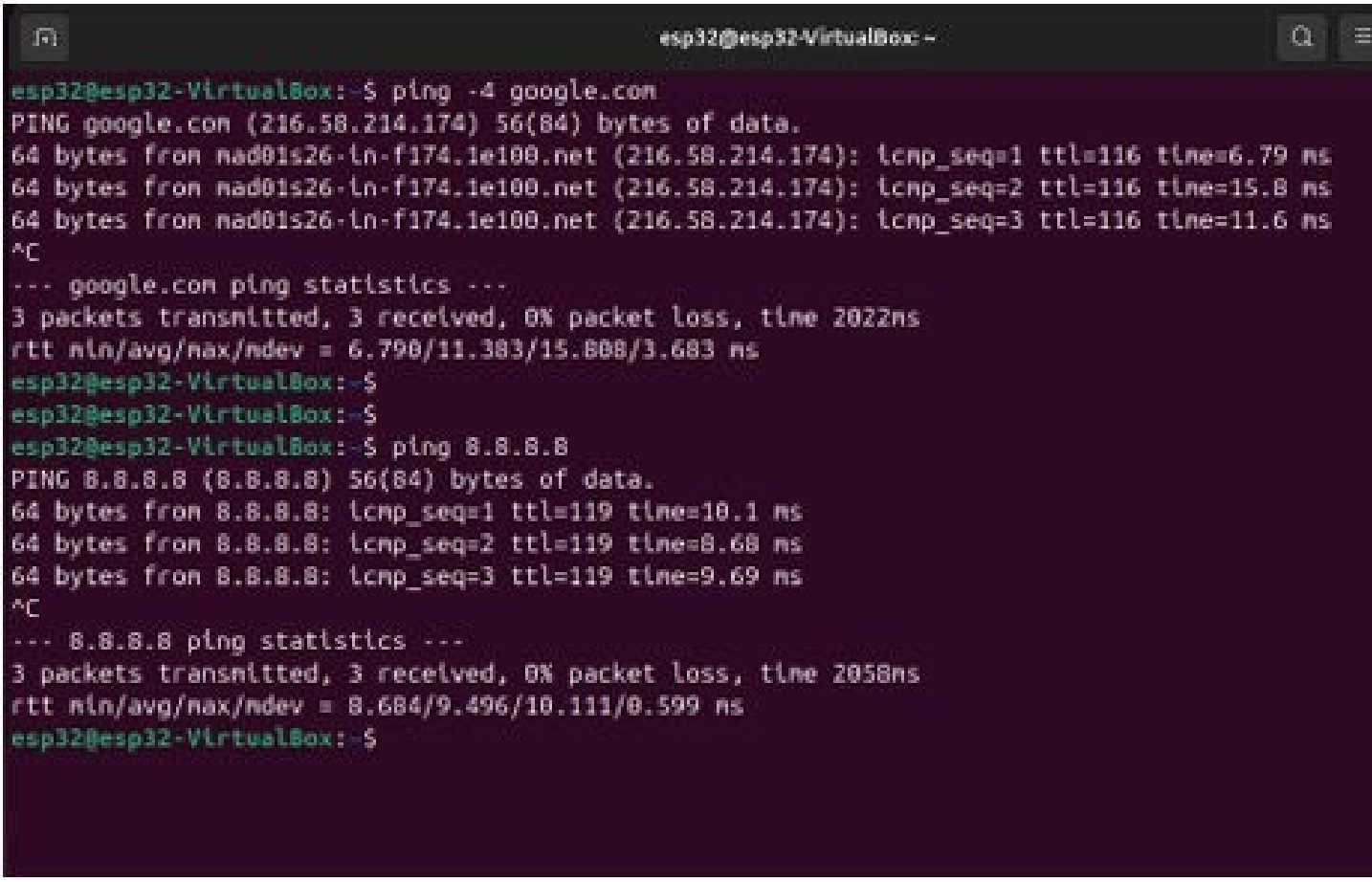

Figure 1 — Test de connectivité réseau (ping google.com et 8.8.8.8)

3. Installation et configuration du pare-feu UFW

Après l'installation de SSH, un pare-feu UFW a été configuré afin de réduire la surface d'attaque de la machine virtuelle.

3.1 Installation

Commande : sudo apt install ufw -y

```
esp32@esp32-VirtualBox:~$ sudo apt install ufw -y
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
ufw est déjà la version la plus récente (0.36.2-6).
ufw passé en « installé manuellement ».
Le paquet suivant a été installé automatiquement et n'est plus nécessaire :
  libllvm19
Veuillez utiliser « sudo apt autoremove » pour le supprimer.
0 mis à jour, 0 nouvellement installés, 0 à enlever et 16 non mis à jour.
esp32@esp32-VirtualBox:~$
```

Figure 2 — Installation du pare-feu UFW

3.2 Définition de la politique par défaut

Bloquer tout trafic entrant, autoriser tout trafic sortant :

```
root@esp32-VirtualBox:/home/esp32# ufw default deny incoming
La stratégie par défaut pour le sens « incoming » a été remp
par « deny »
(veillez à mettre à jour vos règles en conséquence)
root@esp32-VirtualBox:/home/esp32# ufw default allow outgoin
La stratégie par défaut pour le sens « outgoing » a été remp
par « allow »
(veillez à mettre à jour vos règles en conséquence)
root@esp32-VirtualBox:/home/esp32#
```

Figure 3 — Politique par défaut UFW (deny incoming / allow outgoing)

3.3 Ouverture des ports SSH et MQTTs

```
root@esp32-VirtualBox:/home/esp32#  ufw allow 22/tcp
Les règles ont été mises à jour
Les règles ont été mises à jour (IPv6)
root@esp32-VirtualBox:/home/esp32# ufw allow 8883/tcp
Les règles ont été mises à jour
Les règles ont été mises à jour (IPv6)
```

Figure 4 — Autorisation des ports 22/TCP (SSH) et 8883/TCP (MQTTs)

3.4 Vérification des règles

```
root@esp32-VirtualBox:/home/esp32# ufw status verbose
État : actif
Journalisation : on (low)
Par défaut : deny (incoming), allow (outgoing), disabled (routed)
Nouveaux profils : skip

Vers                       Action      De
----                       ------      --
22/tcp                     ALLOW IN    Anywhere
8883/tcp                   ALLOW IN    Anywhere
22/tcp (v6)                ALLOW IN    Anywhere (v6)
8883/tcp (v6)              ALLOW IN    Anywhere (v6)
```

Figure 5 — Règles UFW (ufw status verbose)

```
root@esp32-VirtualBox:/home/esp32# sudo ufw status
État : actif

Vers                       Action      De
----                       ------      --
22/tcp                     ALLOW       Anywhere
8883/tcp                   ALLOW       Anywhere
22/tcp (v6)                ALLOW       Anywhere (v6)
8883/tcp (v6)              ALLOW       Anywhere (v6)

root@esp32-VirtualBox:/home/esp32#
```

Figure 6 — Statut du pare-feu UFW actif

Seuls les ports strictement nécessaires sont ouverts : port 22/TCP pour l'administration SSH et port 8883/TCP pour MQTTs. Les règles s'appliquent en IPv4 et IPv6.

4. Installation du broker Mosquitto

4.1 Installation et démarrage

```
root@esp32-VirtualBox:/home/esp32# sudo apt install -y mosquitto mosquitto-clients
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Le paquet suivant a été installé automatiquement et n'est plus nécessaire :
  libllvm19
Veuillez utiliser « sudo apt autoremove » pour le supprimer.
Les paquets supplémentaires suivants seront installés :
  libcjson1 libdlt2 libmosquitto1 libwebsockets19t64
Les NOUVEAUX paquets suivants seront installés :
```

Figure 7 — Installation de Mosquitto et mosquitto-clients

```
root@esp32-VirtualBox:/home/esp32# sudo systemctl status mosquitto
● mosquitto.service - Mosquitto MQTT Broker
     Loaded: loaded (/usr/lib/systemd/system/mosquitto.service; enabled; preset: enabled)
     Active: active (running) since Tue 2025-12-16 13:08:34 CET; 11min ago
       Docs: man:mosquitto.conf(5)
             man:mosquitto(8)
   Main PID: 19354 (mosquitto)
      Tasks: 1 (limit: 4597)
     Memory: 1.0M (peak: 1.5M)
        CPU: 3.492s
     CGroup: /system.slice/mosquitto.service
             └─19354 /usr/sbin/mosquitto -c /etc/mosquitto/mosquitto.conf

déc. 16 13:08:34 esp32-VirtualBox systemd[1]: Starting mosquitto.service - Mosquitto MQTT
déc. 16 13:08:34 esp32-VirtualBox systemd[1]: Started mosquitto.service - Mosquitto MQTT B
root@esp32-VirtualBox:/home/esp32#
```

Figure 8 — Statut Mosquitto : active (running) sur le port 1883

Le broker Mosquitto est correctement démarré et écoute initialement sur le port 1883 (MQTT non chiffré), avant la mise en place du chiffrement TLS.

5. Mise en place du chiffrement TLS (MQTTs)

5.1 Génération des certificats

Une Autorité de Certification (CA) auto-signée est créée pour signer les certificats du serveur. La certification consiste à faire signer une clé publique par une autorité de confiance afin de lier une identité à cette clé.

Création de la CA et du certificat serveur :

```
root@esp32-VirtualBox:/etc/mosquitto/certs# sudo openssl genrsa -out ca.ke
root@esp32-VirtualBox:/etc/mosquitto/certs# ls
ca.key  README
root@esp32-VirtualBox:/etc/mosquitto/certs# sudo openssl req -x509 -new -n
y ca.key -sha256 -days 3650 -out ca.crt
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:fr
State or Province Name (full name) [Some-State]:paris
Locality Name (eg, city) []:paris
Organization Name (eg, company) [Internet Widgits Pty Ltd]:projet esp32 io
Organizational Unit Name (eg, section) []:iot security
Common Name (e.g. server FQDN or YOUR name) []:mqtt.local
Email Address []:
root@esp32-VirtualBox:/etc/mosquitto/certs#
```

Figure 9 — Génération de la CA (ca.key + ca.crt) et du certificat serveur

```
root@esp32-VirtualBox:/etc/mosquitto/certs# sudo openssl genrsa -out serve
48
root@esp32-VirtualBox:/etc/mosquitto/certs# sudo openssl req -new -key ser
-out server.csr
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:fr
State or Province Name (full name) [Some-State]:paris
Locality Name (eg, city) []:paris
Organization Name (eg, company) [Internet Widgits Pty Ltd]:projet esp32 io
Organizational Unit Name (eg, section) []:MQTT Brocker
Common Name (e.g. server FQDN or YOUR name) []:192.168.1.183
Email Address []:

Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:
An optional company name []:
root@esp32-VirtualBox:/etc/mosquitto/certs# sudo openssl x509 -req -in ser
-CA ca.crt -CAkey ca.key -CAcreateserial \
  -out server.crt -days 3650 -sha256
Certificate request self-signature ok
subject=C = fr, ST = paris, L = paris, O = projet esp32 iot, OU = MQTT Bro
 = 192.168.1.183
root@esp32-VirtualBox:/etc/mosquitto/certs#
```

Figure 10 — Signature du certificat serveur par la CA

5.2 Rôle de chaque fichier

Fichier	Rôle	Confidentialité
ca.key	Clé privée de la CA	NE QUITTE JAMAIS le serveur
ca.crt	Certificat public de la CA	Copier sur les clients
server.key	Clé privée du broker (chiffre TLS)	Reste sur le serveur
server.csr	Requête de signature	Intermédiaire
server.crt	Certificat signé, prouve l'identité du broker	Utilisé par Mosquitto

5.3 Vérification des certificats

pour vérifier les clefs et les certifications:

```
root@esp32-VirtualBox:/etc/mosquitto/certs# openssl x509 -in server.crt -noout -subject -issuer
subject=C = fr, ST = paris, L = paris, O = projet esp32 iot, OU = MQTT Brocker, CN = 192.168.1.
issuer=C = fr, ST = paris, L = paris, O = projet esp32 iot, OU = iot security, CN = mqtt.local
root@esp32-VirtualBox:/etc/mosquitto/certs#
```

pour vérifier la chaîne de confiance et la date (valable 10 ans):

```
root@esp32-VirtualBox:/etc/mosquitto/certs# openssl verify -CAfile ca.crt server.crt
server.crt: OK
root@esp32-VirtualBox:/etc/mosquitto/certs# openssl x509 -in server.crt -noout -dates
notBefore=Dec 16 13:19:13 2025 GMT
notAfter=Dec 14 13:19:13 2035 GMT
root@esp32-VirtualBox:/etc/mosquitto/certs#
```

Figure 11 — Vérification de la chaîne de confiance (valide 10 ans, jusqu'en 2035)

5.4 Configuration Mosquitto TLS

Contenu du fichier /etc/mosquitto/conf.d/secure.conf :

```
root@esp32-VirtualBox:/etc/mosquitto# cd conf.d/
root@esp32-VirtualBox:/etc/mosquitto/conf.d# ls
README

root@esp32-VirtualBox:/etc/mosquitto/conf.d# cat secure.conf
# Désactiver MQTT en clair
port 0

# MQTT sécurisé (TLS)
listener 8883

cafile /etc/mosquitto/certs/ca.crt
certfile /etc/mosquitto/certs/server.crt
keyfile /etc/mosquitto/certs/server.key

allow_anonymous false
root@esp32-VirtualBox:/etc/mosquitto/conf.d#
```

Figure 12 — Configuration Mosquitto TLS (port 0 = désactivé, listener 8883 = TLS uniquement)

5.5 Permissions des fichiers

```
root@esp32-VirtualBox:/etc/mosquitto# sudo chown -R mosquitto:mosquitto /etc/mosquitto/c
root@esp32-VirtualBox:/etc/mosquitto# sudo chmod 600 /etc/mosquitto/certs/server.key
root@esp32-VirtualBox:/etc/mosquitto# sudo chmod 644 /etc/mosquitto/certs/*.crt
root@esp32-VirtualBox:/etc/mosquitto# sudo chown -R mosquitto:mosquitto /var/log/mosquit
root@esp32-VirtualBox:/etc/mosquitto# sudo chmod 755 /var/log/mosquitto
```

Figure 13 — Application des permissions sur les certificats et logs Mosquitto

Mosquitto applique un mécanisme de fail-secure : si les fichiers de sécurité configurés ne sont pas accessibles par l’utilisateur mosquitto, le broker refuse de démarrer.

6. Authentification et contrôle d'accès (ACL)

6.1 Création de l'utilisateur MQTT

Création de l'utilisateur esp32 avec mot de passe hashé (algorithme PBKDF2) :

```
root@esp32-VirtualBox:/etc/mosquitto# sudo mosquitto_passwd -c /etc/mo
o/passwd esp32
Password:
Reenter password:
root@esp32-VirtualBox:/etc/mosquitto#
```

on vérifie si le mdp est bien dans le fichier et est hashe:

```
root@esp32-VirtualBox:/etc/mosquitto# sudo cat /etc/mosquitto/passwd
esp32:$7$101$SX8lRP4l0O6lQhRy$aSeDJ4DEmBH10+8I5X87mlARlXzJzDf5C1US5XTv
2eXFsg0c92BgPLbI6VQm2RrtD5U/krAJYXDEixEw==
root@esp32-VirtualBox:/etc/mosquitto#
```

Figure 14 — Création de l'utilisateur MQTT et vérification du hash dans le fichier passwd

6.2 Configuration finale secure.conf (avec auth + ACL)

Le fichier de configuration final inclut le fichier de mots de passe et le fichier ACL :

```
root@esp32-VirtualBox:/etc/mosquitto# sudo nano /etc/mosquitto/conf.d/
.conf
root@esp32-VirtualBox:/etc/mosquitto# cat /etc/mosquitto/conf.d/secure
# MQTT sécurisé (TLS)
listener 8883

cafile /etc/mosquitto/certs/ca.crt
certfile /etc/mosquitto/certs/server.crt
keyfile /etc/mosquitto/certs/server.key

allow_anonymous false

password_file /etc/mosquitto/passwd
acl_file /etc/mosquitto/acl
root@esp32-VirtualBox:/etc/mosquitto#
```

Figure 15 — Configuration complète : TLS + authentification + ACL

L’utilisateur esp32 peut lire et écrire uniquement sur les topics esp32/123/#. L’accès anonyme est désactivé.

7. Tests depuis le PC Windows

7.1 Récupération du certificat CA

Le certificat ca.crt est copié depuis la VM vers le poste Windows via SCP. Sans ce certificat, la connexion TLS échouerait car la CA est auto-signée.

```
PS C:\Users\karab> scp esp32@192.168.1.183:/etc/mosquitto/certs/ca.cr
esp32@192.168.1.183's password:
Permission denied, please try again.
esp32@192.168.1.183's password:
ca.crt                                      100% 1375   111.9KB/s   00
PS C:\Users\karab>
```

Figure 16 — Copie de ca.crt depuis la VM vers le PC Windows (commande scp)

7.2 Test publish/subscribe depuis Windows

Deux terminaux PowerShell sont utilisés : un pour s'abonner (récepteur), un pour publier (rémetteur).

Terminal 1 (Subscribe) : .mosquitto_sub.exe -h 192.168.1.183 -p 8883 --cafile ca.crt -u esp32 -P esp32 -t esp32/123/status -v

Terminal 2 (Publish) : .mosquitto_pub.exe -h 192.168.1.183 -p 8883 --cafile ca.crt -u esp32 -P esp32 -t esp32/123/status -m "Test MQTTs OK"

8. Connexion de l'ESP32 en MQTTs

8.1 Copie du certificat ca.crt sur l'ESP32

Via Thonny (Affichage → Fichiers), le certificat ca.crt est glissé depuis le PC vers la mémoire flash de l'ESP32 :

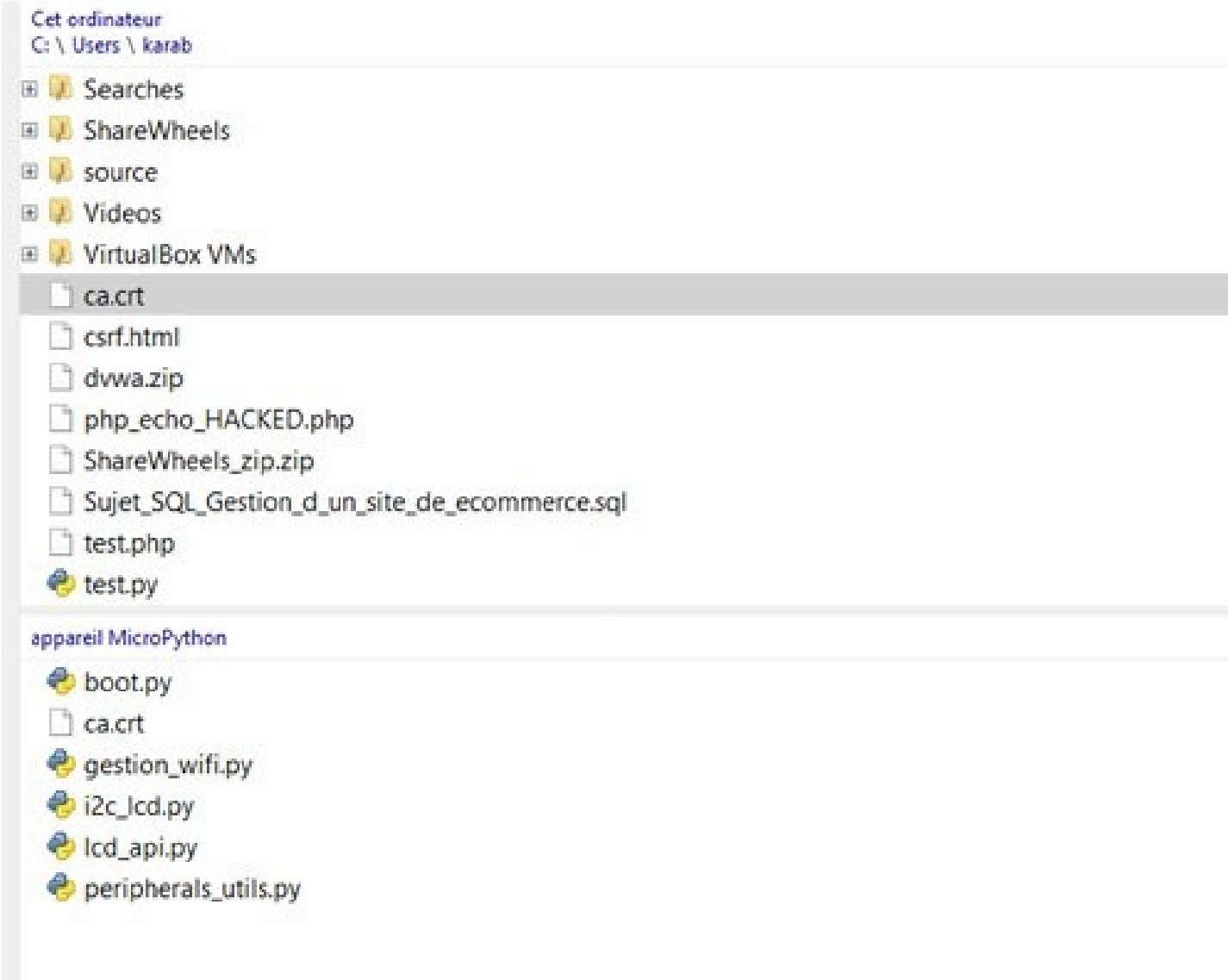

Figure 17 — Arborescence Thonny : ca.crt visible sur l'appareil MicroPython

8.2 Vérification dans les logs Mosquitto

Analyse des logs du broker après connexion de l'ESP32 et vérification que l'utilisateur est bien restreint (sans TLS : protocol error) :

vérification de la connection a mosquitto dans les logs :

sudo tail -f /var/log/mosquitto/mosquitto.log

```
esp32@esp32-VirtualBox:~$ sudo tail -f /var/log/mosquitto/mosquitto.log
1766094042: mosquitto version 2.0.18 terminating
1766094042: Saving in-memory database to /var/lib/mosquitto//mosquitto.db.
1766094042: mosquitto version 2.0.18 starting
1766094042: Config loaded from /etc/mosquitto/mosquitto.conf.
1766094042: Opening ipv4 listen socket on port 8883.
1766094042: Opening ipv6 listen socket on port 8883.
1766094042: mosquitto version 2.0.18 running
1766094102: New connection from 192.168.1.55:61057 on port 8883.
1766094102: New client connected from 192.168.1.55:61057 as esp32_123 (p2, c1, k60, u'esp
1766094198: Client esp32_123 has exceeded timeout, disconnecting.
```

vérifier que l' utilisateur est bien restreint:

```
PS C:\Program Files\mosquitto> .\mosquitto_sub -h 192.168.1.183 -p 8883 -u esp32 -P esp32 -t te
Error: protocol error
PS C:\Program Files\mosquitto>
```

Figure 18 — Logs Mosquitto : connexion ESP32 sur port 8883, et test de restriction ACL

8.3 Code MicroPython et test final

Le code MicroPython établit la connexion Wi-Fi, synchronise l'horloge via NTP (obligatoire pour TLS), puis se connecte au broker en MQTTs. Le message ESP32 CONNECTED TLS OK est reçu côté client Windows :

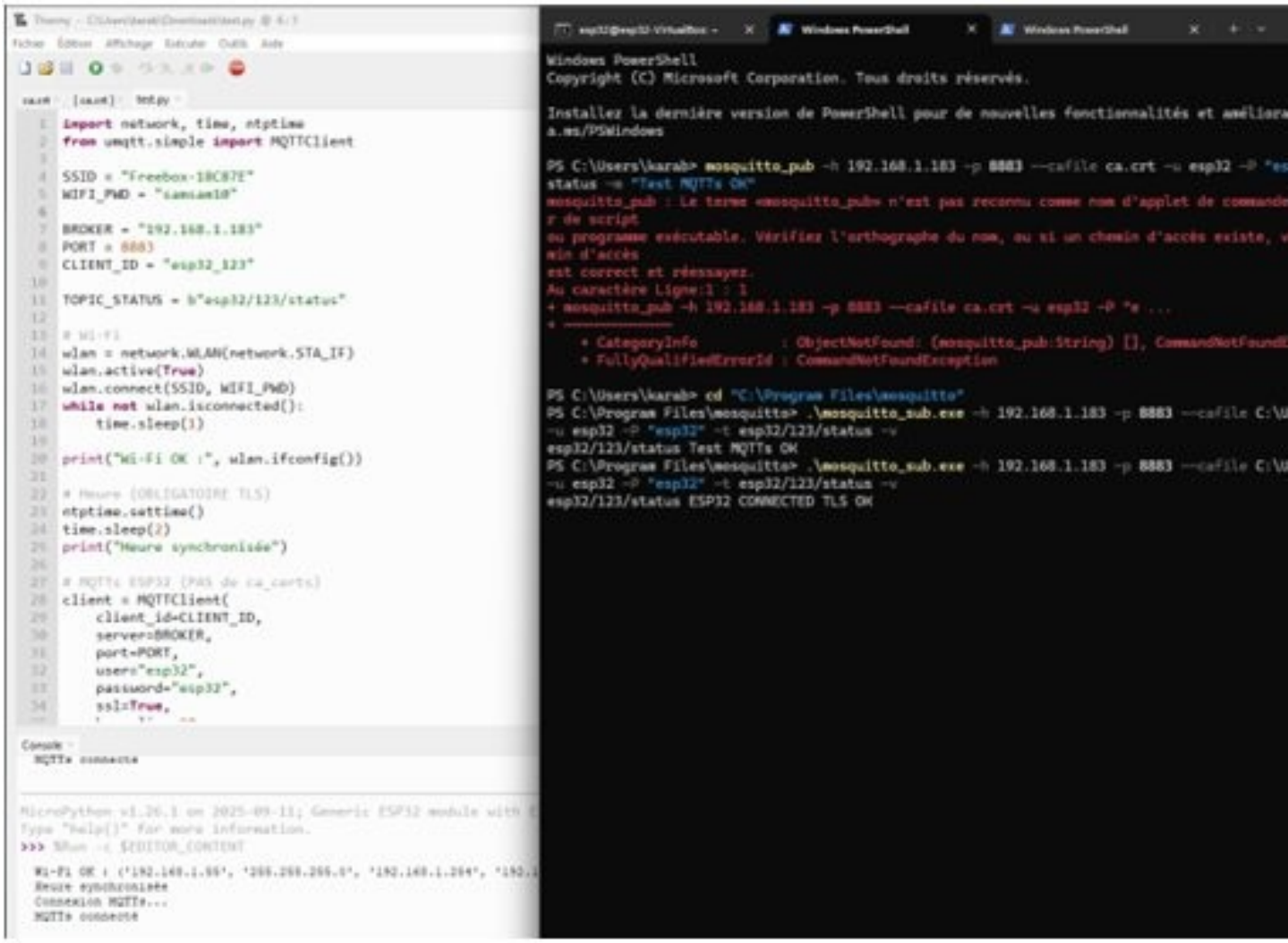

Figure 19 — Code Thonny + réception du message TLS sur le PC Windows

Point critique : la synchronisation NTP via ntptime.settime() est obligatoire avant toute connexion TLS. La validation des certificats dépend de l’horloge système.

9. Bilan et résultats

Étape	Statut
VM Ubuntu en mode bridge, IP 192.168.1.183	✅ OK
SSH fonctionnel depuis Windows	✅ OK
Pare-feu UFW configuré (ports 22 et 8883 uniquement)	✅ OK
Mosquitto installé et démarré	✅ OK
Certificats CA et serveur générés (valides 10 ans)	✅ OK
Mosquitto TLS uniquement (port 8883, port 1883 désactivé)	✅ OK
Authentification username/password activée	✅ OK
ACL restreignant les topics par utilisateur	✅ OK
Test publish/subscribe validé depuis Windows	✅ OK
ESP32 connecté au broker en MQTTs (TLS)	✅ OK
Message ESP32 CONNECTED TLS OK reçu côté client Windows	✅ OK

Garanties de sécurité obtenues

Propriété	Mécanisme
Confidentialité	Toutes communications chiffrées TLS (port 8883). Port 1883 désactivé.
Authenticité	Certificat serveur signé par la CA, vérifié par les clients avec ca.crt.
Intégrité	TLS garantit que les messages ne peuvent pas être altérés en transit.
Contrôle d'accès	Authentification user/password + ACL restreignant les topics autorisés.
Cloisonnement	Pare-feu UFW : tout trafic entrant bloqué sauf SSH et MQTTs.

Conclusion : L'architecture mise en place garantit la confidentialité, l'intégrité et l'authenticité des communications entre l'ESP32 et le serveur MQTT, sans exposer directement l'objet connecté sur le réseau.

Nissa Karadag — TP IoT Sécurisé, Décembre 2025